Título del TFM

Nombre del autor

September 18, 2026

Contents

1 Basic operation **3**

1.1 Screens and settings . . . 3

1.1.1 Main screen . . . 3

1.1.2 Jog screen . . . 4

1.1.3 Profile screen . . . 8

1.1.4 Optical alignment screen . . . 10

2 Routines **14**

2.1 RasterScan . . . 14

2.1.1 `clean_profile_data()` . . . 15

2.1.2 `save_profile_txt()` . . . 16

2.1.3 `create_heatmap()` . . . 17

2.2 Automaper . . . 19

2.2.1 Building blocks . . . 19

2.2.2 Excel-Based Chip Description . . . 20

2.2.3 checkPositionsAutomap . . . 22

2.2.4 moveAutomapOPM . . . 23

2.2.5 Workflow . . . 23

2.2.6 Relevant Configuration . . . 24

2.2.7 Results . . . 24

2.2.8 Precautions . . . 24

2.2.9 moveAutomapOPM_OSA . . . 24

2.2.10 Workflow . . . 24

2.2.11 Relevant Configuration . . . 25

2.2.12 Results . . . 25

2.2.13 Precautions . . . 25

A TwinCAT XAE (eXtended Automation Engineering) Installation **26**

B Suruga Seiki API Documentation **32**

B.1 Profile class 32
B.2 System class 34

Chapter 1

Basic operation

1.1 Screens and settings

1.1.1 Main screen

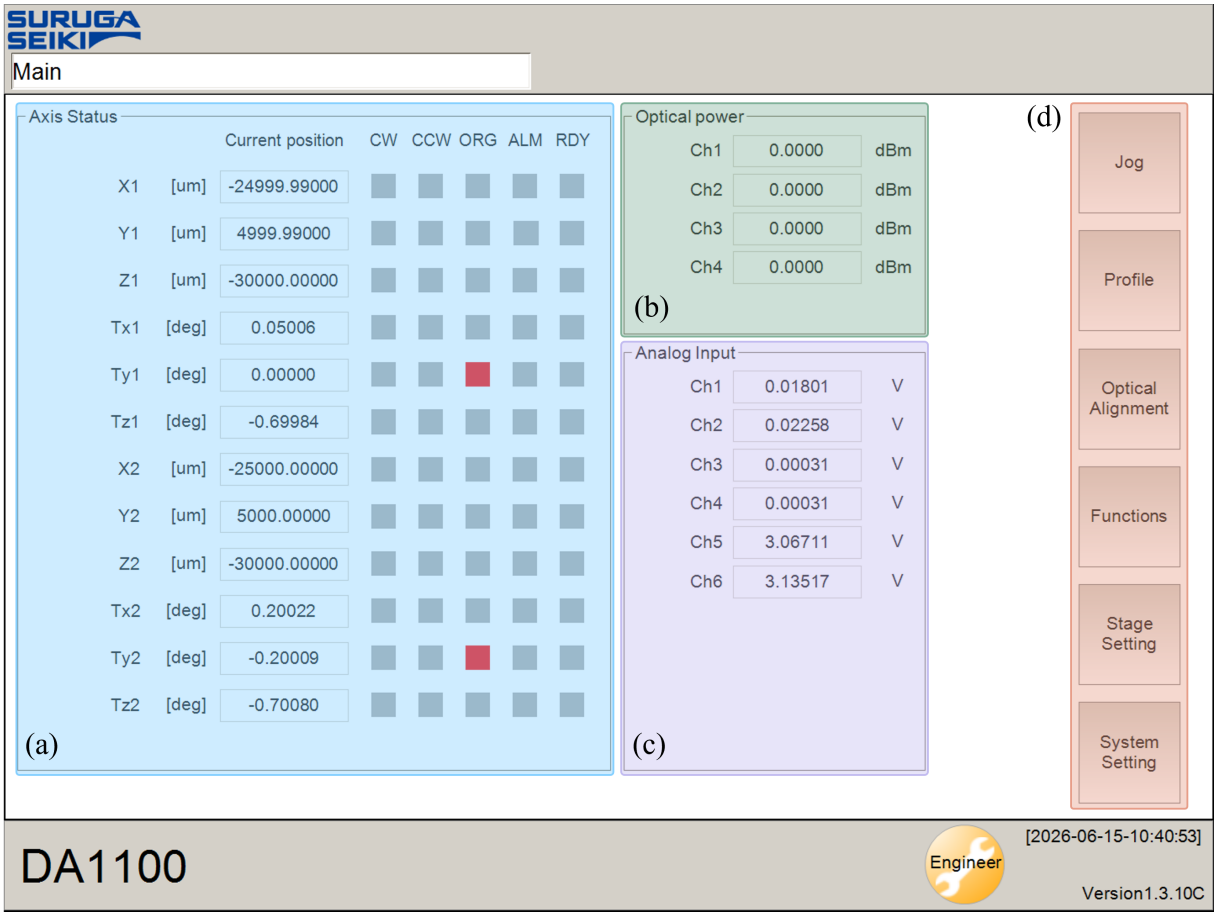


Figure 1.1: Main screen of the Suruga positioning system. The interface is divided into four main sections: (a) Axis Status, which displays the current position and status of each axis; (b) Optical Power, which displays the optical power measured by the connected power meter; (c) Analog Input, which displays the voltage measured at each available analog input channel; and (d) Function Access, which provides direct access to the main functionalities and configuration options of the Suruga platform.

This screen is divided into three main sections. The first section, Axis Status, allows the user to monitor the current position of each axis, the CW and CCW limit switch status, the origin sensor status, the alarm outputs, and the availability of each servo. As listed below:

- **Current Position:** Displays the current coordinates of each axis. Linear axes are displayed in micrometers (μm), while rotational axes are displayed in degrees ($^\circ$). Axes with the number 1 correspond to the left arm of the Suruga positioning system, while axes with the number 2 correspond to the right arm.
- **CW:** Displays the status of the clockwise (CW) limit sensor. Red indicates that the limit has been reached, while gray indicates that the axis is within the safe operating range.
- **CCW:** Displays the status of the counterclockwise (CCW) limit sensor. Red indicates that the limit has been reached, while gray indicates that the axis is within the safe operating range.
- **ORG:** Displays the status of the origin sensor. For this particular stage, no origin sensor is available, so the indicator will remain continuously displayed in red or gray.
- **ALM:** Indicates the alarm status of the axis. Red is displayed when an alarm occurs, such as an overload or a similar fault condition. Gray indicates normal operation.
- **RDY:** Displays the operation status of each axis. Green indicates that the servo is enabled and ready for operation, while gray indicates that the servo is turned off.

The second section, Optical Power, displays the current reading of the power meter, if one is available. Finally, the Analog Input section allows the user to monitor the values of the different analog inputs. On the left side of the screen, direct access to the different functionalities of the Suruga platform is provided.

1.1.2 Jog screen

By pressing the Jog button in the quick access section, indicated in part (d) of the main screen in Fig. 1.1(d), the user is directed to the Jog screen shown in Fig. 1.2.

This screen allows the user to perform movements along the different axes of the stage and to adjust the rotation angles of the corresponding axes. It is important to note that, once the Suruga Seiki stage is powered on, no power is supplied to the motors by default. Therefore, the Servo button on this screen must be pressed to enable each axis of the stage before any movement can be performed.



Figure 1.2: Jog screen of the Suruga positioning system. The interface is divided into three main sections: (a) Measurement Monitoring, which displays the optical power and analog input measurements; (b) Movement Control, which provides the servo, error reset, movement mode, velocity, and stop controls; and (c) Axis Control, which provides independent movement and homing controls for each axis of both arms, together with the corresponding position, velocity, limit switch, and alarm status information.

The jog screen can be divided into three sections. Section (a) Measurement Monitoring, displays the optical power measured by the power meter, as well as the voltage measured through the analog inputs. Section (b) Movement Control, provides access to five control buttons and selectors:

- **Servo:** Enables or disables the servos of the stage.
- **Error Reset:** Resets the servo alarm in case an alarm occurs.
- **Mode:** Allows the user to select between the three available movement modes: Jog, Inching, and Target. In Jog mode, the movement continues while the movement button is held down. In Inching mode, pressing a movement button moves the selected axis by the distance specified in the Inching Distance parameter. In Target mode, the selected axis moves to the target position specified in Section (c).
- **Velocity** – Allows the user to select between the three preset speed configurations: High, Mid, and Low.
- **Stop** – Stops the movement of the selected axis.

Finally, Section (c) Axis Control, is divided into twelve subsections, six for each arm. Each subsection corresponds to one of the six degrees of freedom available for the respective arm:

translational movements along the X, Y, and Z axes, and angular movements around the X, Y, and Z axes (T_x, T_y, and T_z, respectively). Each of the subsections in Section (c) provides a set of movement control buttons, as well as fields for defining the target position and the inching distance. The user can also define the movement velocity, while the current position of the corresponding axis is continuously displayed. In addition, each subsection includes status indicators for the CW, CCW, and ORG limit switches, as well as an ALM indicator.

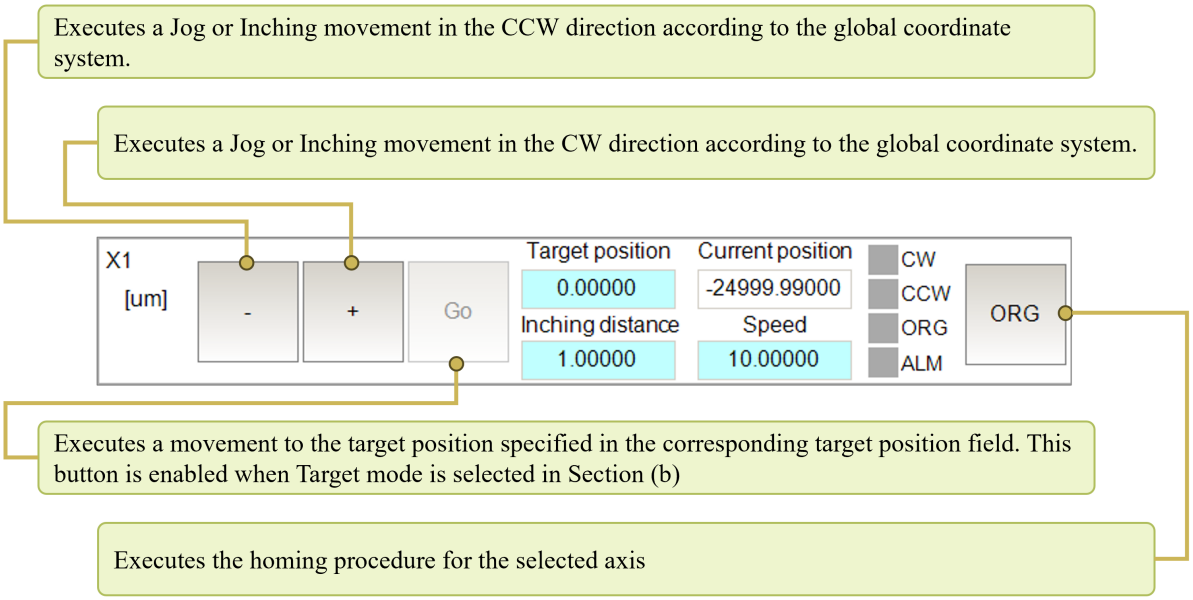


Figure 1.3: Movement controls and status indicators for an individual stage axis, including target position, inching distance, velocity, current position, limit-switch status, alarm status, and homing control.

The following figures illustrate the movement directions available for the three translational degrees of freedom of the stage. The translational movements along the X, Y, and Z axes are presented, showing the directions associated with the + and − buttons for each arm.

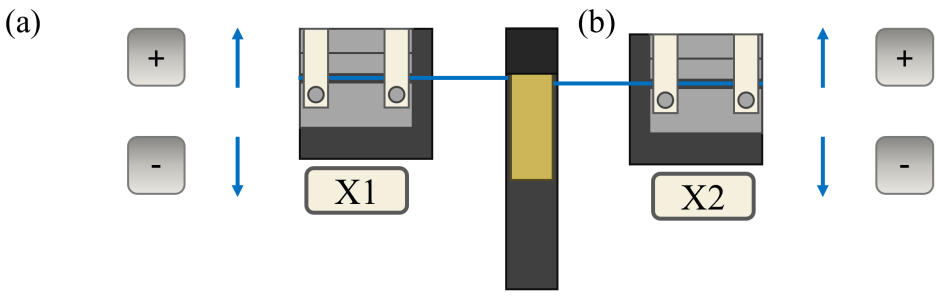


Figure 1.4: Movement directions along the X axis for arm 1 (a) and arm 2 (b) associated with the + and − buttons.

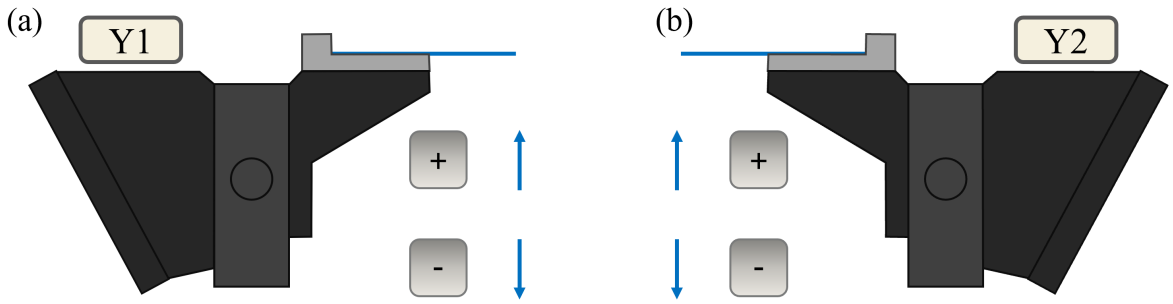


Figure 1.5: Movement directions along the Y axis for arm 1 (a) and arm 2 (b) associated with the + and − buttons.

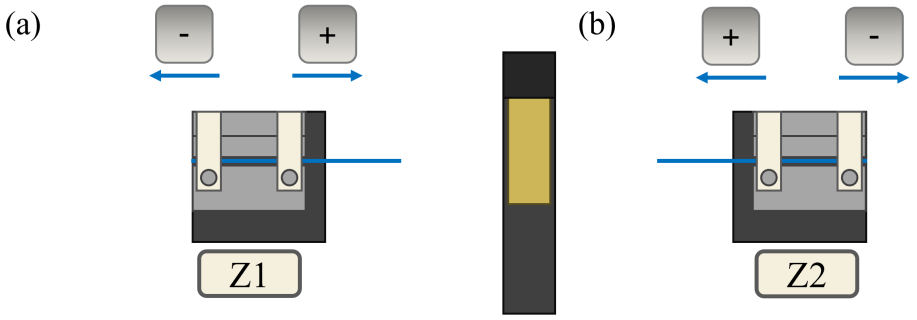


Figure 1.6: Movement directions along the Z axis for arm 1 (a) and arm 2 (b) associated with the + and − buttons.

The corresponding angular movements are presented next. These figures illustrate the rotation directions associated with the + and − buttons for rotations around the X, Y, and Z axes (T_x, T_y, and T_z, respectively), for both arms of the stage.

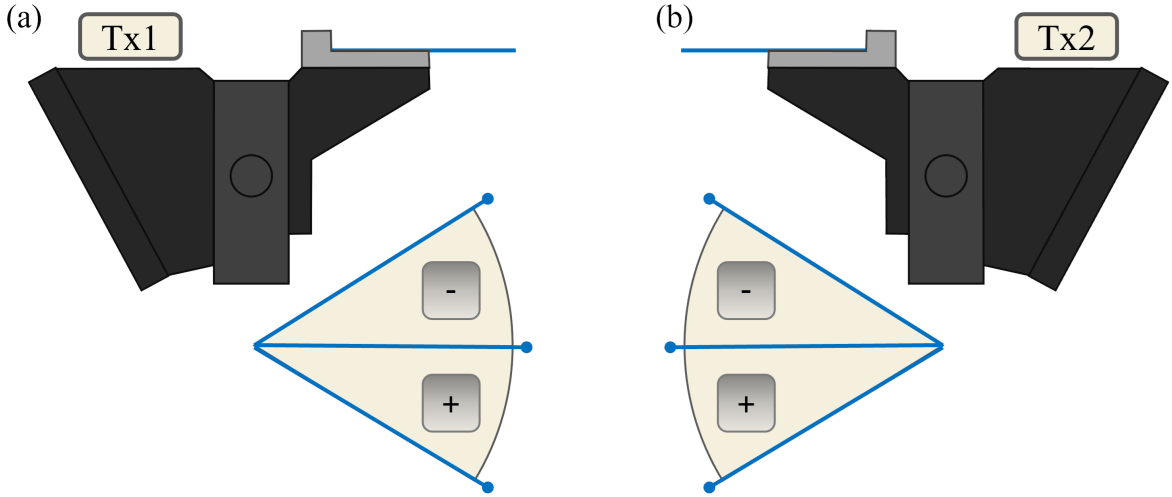


Figure 1.7: Rotation directions around the X axis produced by the + and − buttons for arm 1 (a) and arm 2 (b).

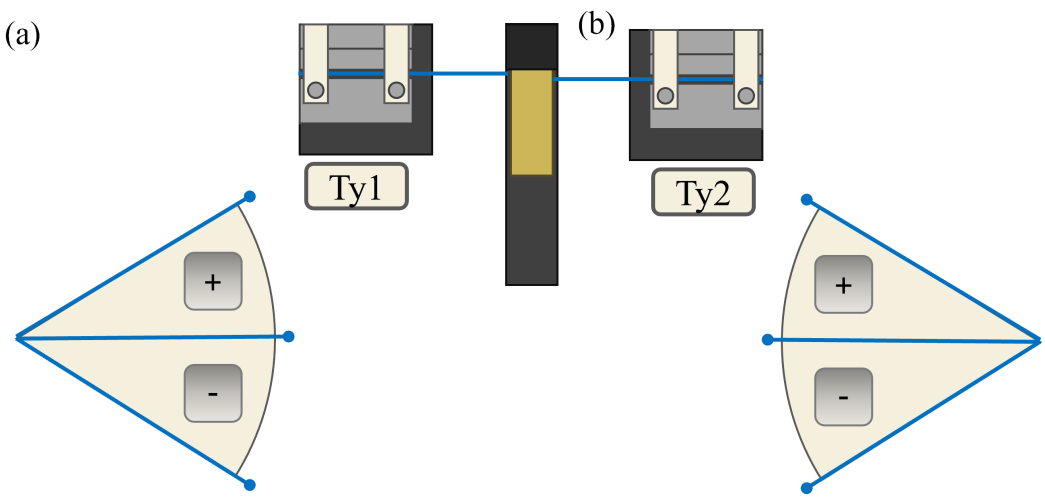


Figure 1.8: Pendiente.

Figure 1.9: Rotation directions around the Y axis produced by the + and − buttons for arm 1 (a) and arm 2 (b).

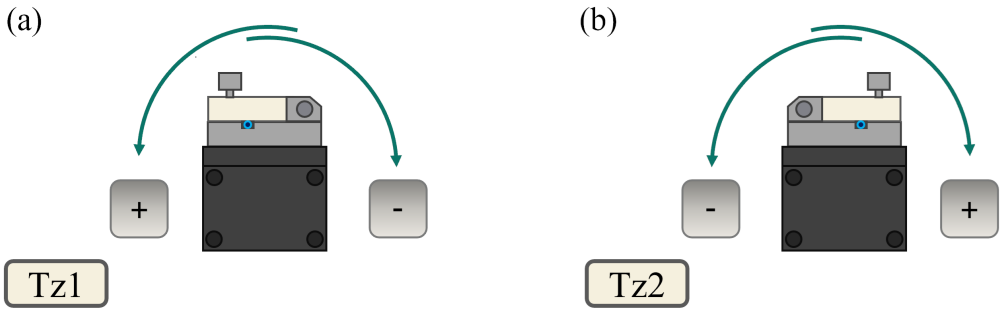


Figure 1.10: Rotation directions around the Z axis produced by the + and − buttons for arm 1 (a) and arm 2 (b).

1.1.3 Profile screen

The SurugaSeiki.Motion.Profile class provides a high-level interface for configuring, executing, and monitoring profiling operations on the Suruga Seiki platform. To use this functionality, an instance of the Profile class is created, providing access to a set of methods for configuring the acquisition parameters, starting and stopping the profiling process, monitoring its status, and requesting the acquired data.

The class also contains several nested types that complement its functionality. ProfileParameter is used to define the parameters required to configure a profiling operation, while ProfileData represents the data acquired during the operation. In addition, the Status and ErrorCode enumerations are used to identify, respectively, the current status of the profiling process and the error conditions that may occur during its execution.

For a detailed description of the functionalities, methods, and data structures of the Profile class, refer to the class definition provided in Appendix B.1. However in this section we will talk about the Profile screen.



Figure 1.11: Pendiente.

1.1.4 Optical alignment screen

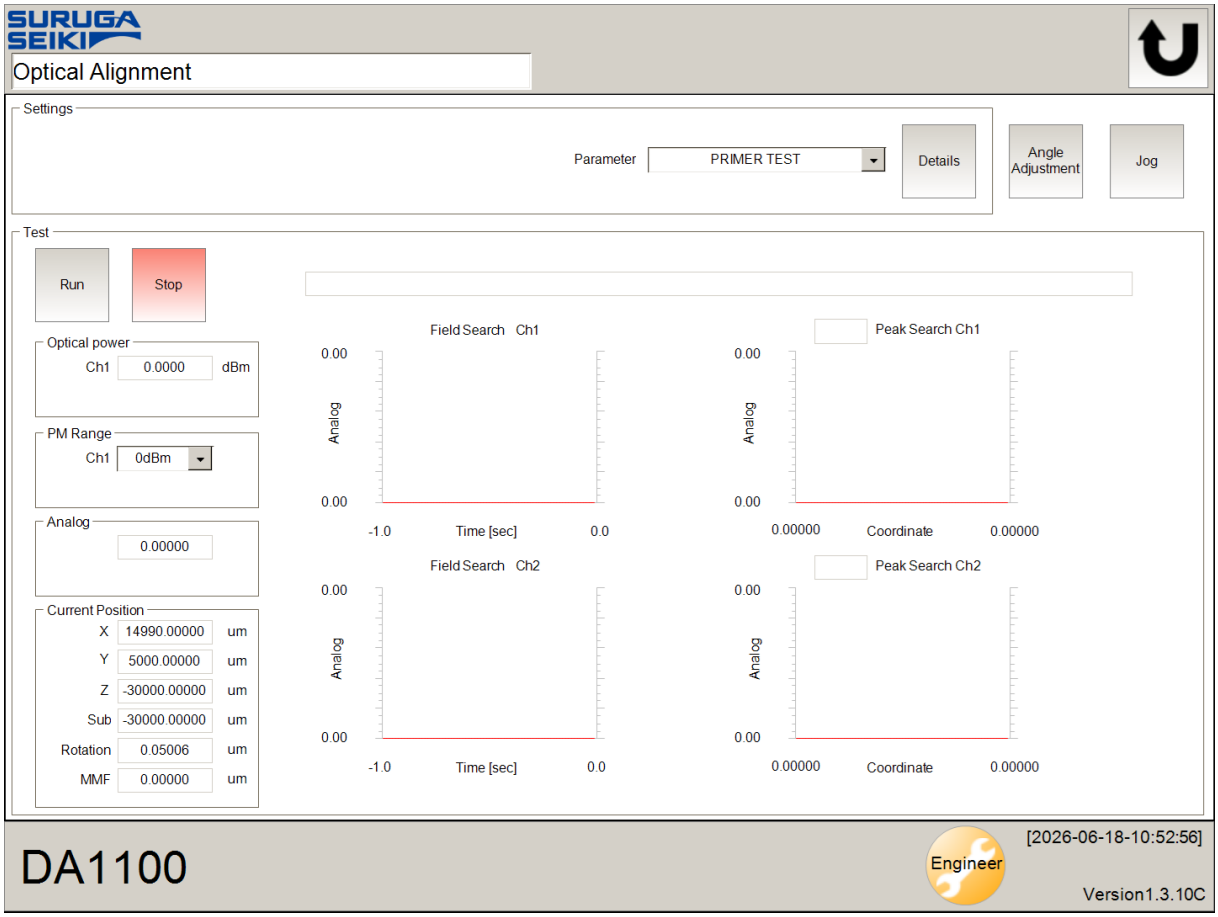


Figure 1.12: Pendiente.

El usuario puede llevar a cabo los siguientes métodos de alineamiento:

- **Alineamiento Single (Single Alignment):** Realiza el alineamiento utilizando un único eje.
- **Alineamiento Flat (Flat Alignment):** Realiza el alineamiento en un plano utilizando dos ejes.
- **Alineamiento Focus (Focus Alignment):** Además del alineamiento Flat, realiza el ajuste sobre el eje óptico empleando tres ejes.
- **Alineamiento por rotación de dos canales (2ch Rotation Alignment):** Realiza un alineamiento Flat utilizando dos señales ópticas y corrige el ángulo alrededor del eje óptico en función de la posición y de la distancia configurada.
- **Alineamiento MMF (MMF Rotation Alignment):** Posiciona una fibra multimodo (MMF, Multi-Mode Fiber) en el lado de salida y corrige la posición y el ángulo del arreglo de fibras en el lado de entrada.

Alineamiento Flat (Flat Alignment):

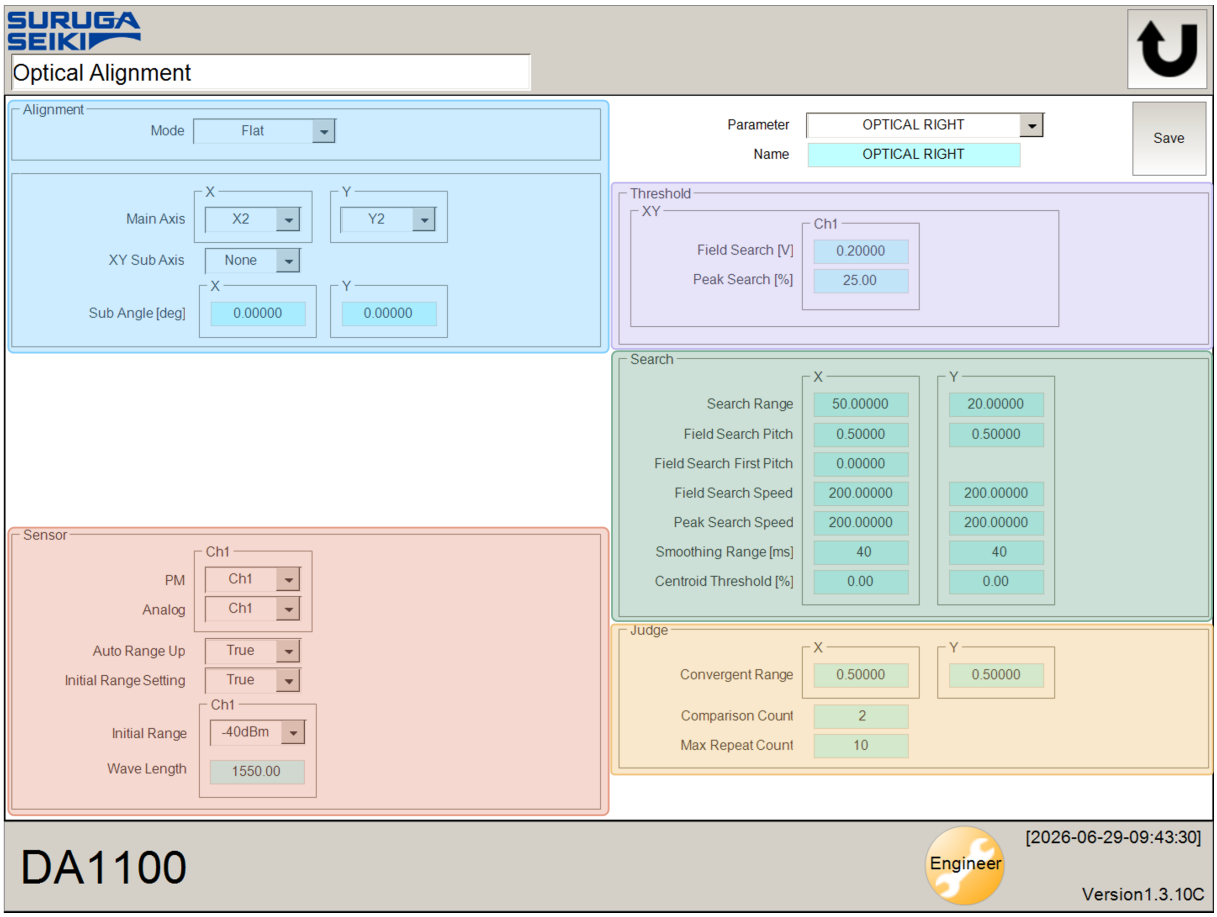


Figure 1.13: Pendiente.

Se parte de la condición en la que la fibra y el Device Under Test (DUT) se encuentran desalineados, por lo que el sistema desconoce la ubicación del punto de máxima potencia óptica. Debido a ello, no es posible iniciar directamente un Peak Search, ya que este requiere partir de una señal detectable para localizar el máximo.

Como primer paso, el algoritmo ejecuta un Field Search, cuyo objetivo es explorar el área alrededor de la posición inicial hasta encontrar una potencia lo suficientemente grande. La condición que determina el éxito de esta etapa es:

$$P_{\text{OPM}} > \text{FieldSearchThreshold} \tag{1.1}$$

Donde FieldSearchThreshold corresponde al nivel mínimo de potencia, expresada en voltios haciendo uso de la salida analógica del OPM, que indica que el sistema ha encontrado una señal válida.

Durante esta etapa, el movimiento sigue una trayectoria en espiral cuadrada que se expande progresivamente alrededor del punto inicial. En cuanto la potencia medida supera el umbral establecido, el algoritmo abandona el Field Search e inicia el proceso de Peak Search, encargado de localizar con precisión el punto de máxima potencia.

Si la trayectoria alcanza los límites definidos por SearchRangeX o SearchRangeY sin encontrar una señal superior a FieldSearchThreshold, el proceso finaliza con el estado FieldSearchRangeOver, indicando que no fue posible localizar una señal dentro del área de búsqueda especificada.

El comportamiento del algoritmo empleado para un alineamiento tipo flat está determinado por un conjunto de parámetros que definen el área de búsqueda, la resolución del barrido y el criterio para iniciar la búsqueda fina. En la Tabla 1.1 se describen los parámetros más relevantes en conjunto con su equivalente en el DLL provisto por Suruga Seiki.

Table 1.1: Parámetros de configuración

Parámetro	**Descripción**	**Tipo .NET**
Alignment		
mainStageNumberX	Número del eje principal utilizado para el movimiento en X.	UInt16
mainStageNumberY	Número del eje principal utilizado para el movimiento en Y.	UInt16
subStageNumberXY	Número del sub-eje utilizado para el movimiento en XY.	UInt16
subAngleX	Ángulo del sub-eje para la alineación en X.	Double
subAngleY	Ángulo del sub-eje para la alineación en Y.	Double
Sensor		
pmCh	Canal del medidor de potencia utilizado durante la alineación.	UInt16
analogCh	Canal de entrada analógica utilizado para la medición.	UInt16
pmAutoRangeUpOn	Habilita el incremento automático del rango del medidor de potencia.	Boolean
pmInitRangeSettingOn	Habilita el uso de un rango inicial para el medidor de potencia.	Boolean
pmInitRange	Valor del rango inicial del medidor de potencia.	Int16
Threshold		
fieldSearchThreshold	Umbral utilizado durante la búsqueda de campo.	Double
peakSearchThreshold	Umbral utilizado durante la búsqueda del pico de señal.	Double
Search		
searchRangeX	Rango de búsqueda en la dirección X.	Double
searchRangeY	Rango de búsqueda en la dirección Y.	Double
fieldSearchPitchX	Paso de exploración en X durante la búsqueda de campo.	Double
fieldSearchPitchY	Paso de exploración en Y durante la búsqueda de campo.	Double
fieldSearchFirstPitchX	Primer paso de exploración en X durante la búsqueda de campo.	Double
fieldSearchSpeedX	Velocidad de exploración en X durante la búsqueda de campo.	Double

Continued on next page

Table 1.1: Parámetros de configuración (Continued)

fieldSearchSpeedY	Velocidad de exploración en Y durante la búsqueda de campo.	Double
peakSearchSpeedX	Velocidad de búsqueda del pico en X.	Double
peakSearchSpeedY	Velocidad de búsqueda del pico en Y.	Double
smoothingRangeX	Ventana de suavizado aplicada a los datos en X.	UInt32
smoothingRangeY	Ventana de suavizado aplicada a los datos en Y.	UInt32
centroidThresholdX	Umbral para el cálculo del centroide en X.	Double
centroidThresholdY	Umbral para el cálculo del centroide en Y.	Double
Judge		
convergentRangeX	Rango de convergencia permitido en X.	Double
convergentRangeY	Rango de convergencia permitido en Y.	Double
comparisonCount	Número de comparaciones realizadas para verificar la convergencia.	UInt16
maxRepeatCount	Número máximo de repeticiones permitidas durante el proceso de alineación.	UInt16

Chapter 2

Routines

2.1 RasterScan

The routine performs a two-dimensional raster scan by moving along a selected axis while acquiring a profile along the perpendicular axis at each scan position. For example, when the X axis is selected as the scan axis, the system moves along X according to the specified scan step and acquires a Z-profile at each X position. This approach is particularly useful for vertical coupling measurements, where the optical power distribution can be characterized as a function of the X and Z positions.

During the scan, the profiles acquired using the Suruga Seiki utilities can be saved for further analysis. The acquired data are then cleaned by removing invalid or irrelevant samples that could interfere with the generation of the heatmap. The processed profiles are interpolated onto a common grid and used to generate an interactive heatmap representing the measured power distribution.

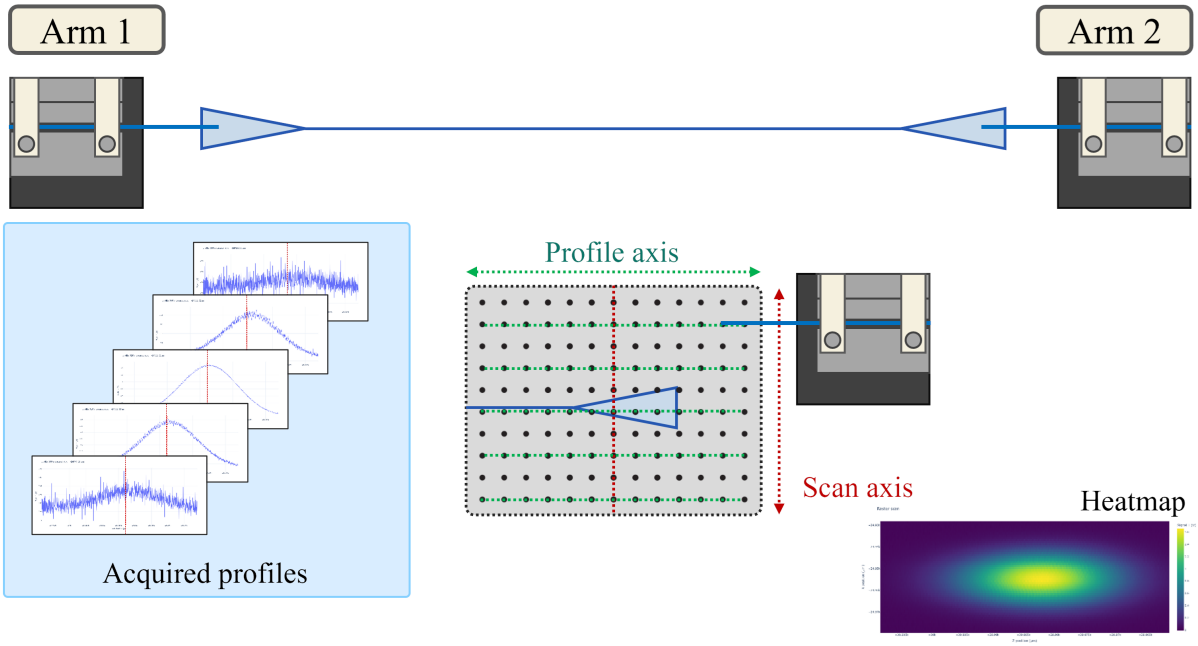


Figure 2.1: Two-dimensional raster scan for optical coupling localization. Profiles are acquired along the profile axis while the stage is incrementally moved along the perpendicular scan axis. The acquired profiles are combined to reconstruct a two-dimensional heatmap, allowing the position of maximum optical coupling to be identified.

Once the scan is complete, the routine identifies the global maximum power measured within the scanned area and, if enabled, automatically moves the arm to the corresponding X and Z position.

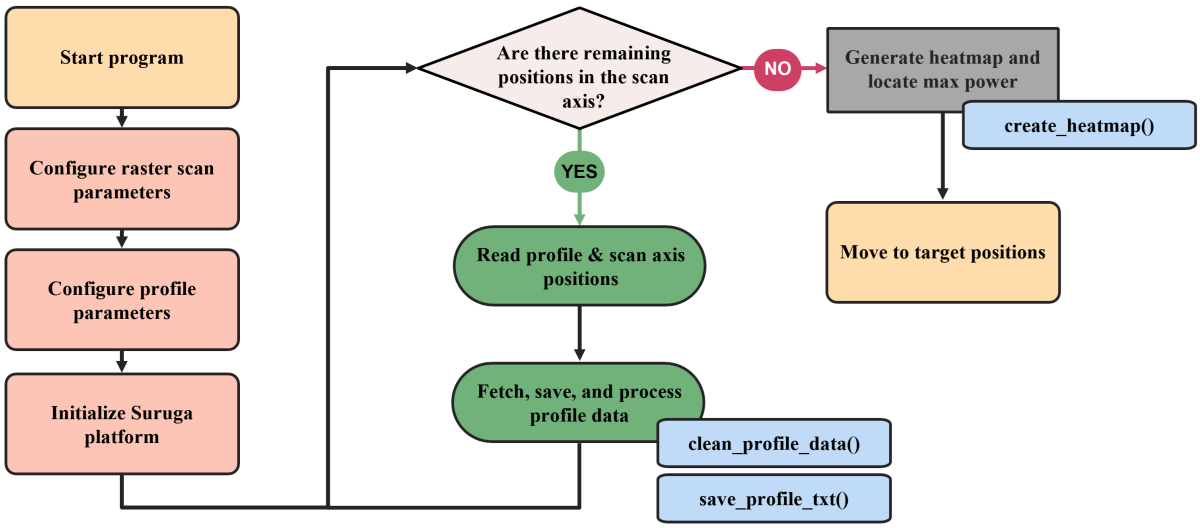


Figure 2.2: Flowchart of the *RasterScan* routine. The process begins with the configuration of the raster scan and profile parameters, followed by the initialization of the Suruga Seiki platform. For each position along the scan axis, the profile and scan-axis positions are obtained, and the acquired data are retrieved, stored, and processed using `clean_profile_data()` and `save_profile_txt()`. The process is repeated until all positions along the scan axis have been completed. Finally, the processed profiles are passed to `create_heatmap()` to reconstruct the two-dimensional power map and locate the maximum signal, after which the system can move to the corresponding target positions.

2.1.1 `clean_profile_data()`

Objective:

Clean and prepare the data acquired by the profile function for further processing.

This function receives the complete dictionary returned by `profile.get_profile_data()` and extracts only the variables required for the raster scan:

- `main_position`: the position of the axis being profiled.
- `signal1`: the measured optical signal.

The function then performs the following operations:

1. Converts the extracted data into NumPy arrays.
2. Removes invalid samples, including:
    - Non-finite position values.
    - Non-finite signal values.
    - Samples where the position is equal to zero.
3. Sorts and reorganizes the remaining samples according to their position.

The function **does not modify or save the original profile data**. Its purpose is to generate a cleaner and more manageable representation of the acquired data, which can then be used for further processing, particularly for generating the heatmap with `create_heatmap()`.

2.1.2 `save_profile_txt()`

Objective:

Preserve the complete profile data acquired during the measurement, together with the experimental parameters required to document and reproduce the measurement.

This function receives the complete dictionary returned by `profile.get_profile_data()`, including the following variables:

- `main_position`: position of the main axis during the profile.
- `sub1_position`: position of the first secondary axis.
- `sub2_position`: position of the second secondary axis.
- `signal1`: first measured signal.
- `signal2`: second measured signal.

In addition to the acquired data, the function constructs a header containing the experimental parameters and the stage positions at the time of acquisition:

- **Date:** Date and time of the measurement.
- **Center X:** X-axis center position.
- **Center Z:** Z-axis center position.
- **Arm:** Selected measurement arm or configuration.
- **Scan axis:** Axis along which the raster scan is performed.
- **Scan width:** Total width of the scan.
- **Scan step:** Distance between consecutive scan positions.
- **Profile axis:** Axis along which the profile is acquired.
- **Profile range:** Total range covered by the profile.
- **Profile speed:** Speed used during profile acquisition.

The function then stores the complete dataset and the associated metadata in a **tab-separated TXT file** using NumPy's `np.savetxt()` function.

Unlike `clean_profile_data()`, this function does **not** remove, filter, or reorganize the acquired samples. Its purpose is to preserve the original measurement data as completely as possible, while providing the experimental metadata necessary to identify and reproduce the measurement.

The resulting TXT file therefore contains both the **raw profile data** and the **measurement parameters** associated with its acquisition.

2.1.3 `create_heatmap()`

Objective:

Reconstruct a two-dimensional signal map from all the profiles acquired during the raster scan, generate an interactive heatmap, and determine the global maximum of `signal1` together with its corresponding X and Z coordinates.

The function receives two inputs:

- `scan_data`: a dictionary containing the processed profiles acquired during the raster scan. Each entry has the following structure:

```
[
    {
        "x": ...,
        "z_positions": ...,
        "signal": ...
    },
    ...
]
```

- `output_path`: the path where the generated heatmap will be saved.

Data Reconstruction

First, the function takes the Z positions from the first acquired profile and uses them as the common reference axis:

```
z_common = scan_data[0]["z_positions"]
```

Since the Z positions may not be identical for every profile, each profile is interpolated onto this common Z-axis. This ensures that all profiles have the same number of samples and can be represented consistently in a two-dimensional matrix.

For each profile, the function:

1. Retrieves the corresponding X position.
2. Retrieves the Z positions and measured signal.
3. Interpolates the signal onto `z_common`.
4. Adds the interpolated signal to the 2D signal matrix.

The resulting matrix represents the measured signal as a function of the X and Z coordinates and is used to generate the heatmap.

Heatmap Generation

The reconstructed 2D signal matrix is used to generate an **interactive Plotly heatmap**, where:

- The **X-axis** represents the raster scan position.
- The **Z-axis** represents the profile position.
- The **color scale** represents the measured `signal1`.

The resulting heatmap provides a two-dimensional visualization of the measured optical signal across the scanned area and is saved to the location specified by `output_path`.

Global Maximum Detection

In addition to generating the heatmap, the function determines the position of the maximum measured signal.

For each profile, the function first identifies its **local maximum**. The local maximum is then compared with the maximum found in the previous profiles. By continuously retaining the highest value, the function ultimately determines the **global maximum** across the entire raster scan.

The global maximum is stored together with its corresponding coordinates:

- **Maximum signal:** maximum value of `signal1`.
- **X coordinate:** X position of the profile where the maximum was found.
- **Z coordinate:** Z position corresponding to the maximum signal within that profile.

Therefore, `create_heatmap()` performs two main tasks: it **reconstructs and visualizes the complete raster scan as a two-dimensional signal map**, and it **identifies the position of the maximum output power** within the scanned region.

2.2 Automaper

Source code: automaper.py.

`automaper.py` provides a set of utilities for loading, interpreting, visualizing, and managing photonic chip layouts described through an Excel-based chip map.

The module acts as an intermediate layer between the logical description of a photonic chip and the measurement software responsible for its physical characterization. Instead of hardcoding device coordinates, port locations, or measurement combinations in Python, the chip structure is defined externally in an Excel workbook. Automaper parses this information and converts it into structured Python data models that can be used by the positioning and measurement routines.

The general workflow is:

```
Excel chip map
      ↓
AUTOMAPER
      ↓
Chip data model
      ↓
Measurement paths
      ↓
Measurement plan
      ↓
Positioning / measurement software
```

The main purpose of automaper.py is therefore measurement-plan generation and management, rather than direct hardware control. The module does not move the positioning stages, acquire optical power, or execute the measurements itself. Instead, it produces the structured information required by other parts of the characterization software to perform these operations.

2.2.1 Building blocks

Automaper can be divided into three functional blocks:

1. **Chip representation:** Loads the device and port definitions from the Excel workbook and converts them into the internal `chip` data structure.
2. **Measurement-path management:** Defines the possible input-to-output connections and allows the user to enable or disable individual measurement paths through the `MeasurementPaths` worksheet.
3. **Measurement-plan generation:** Combines the chip topology with the enabled measurement paths and optional coupling constraints to generate the final `measurement_plan`.

The principal data structures generated by the module are therefore:

- chip
- measurement_paths

- measurement_plan

These structures represent progressively more specific levels of information, from the physical description of the chip to the exact measurements that should be executed.

2.2.2 Excel-Based Chip Description

The Excel workbook is used as the external source for the chip topology. AUTOMAPER expects the following worksheets:

Table 2.1: Worksheets used to define the chip measurement configuration.

Worksheet	**Description**
Metadata	Defines the context of the measurement.
Devices	Defines the devices available for measurement.
Ports	Defines the ports associated with each device, including their position, direction, and coupling type.
MeasurementPaths	Defines the input-to-output measurement paths to be considered.

Metadata worksheet

The `Metadata` worksheet is intended to provide traceability and documentation for the measurement configuration. It allows a workbook to retain contextual information about the chip without mixing descriptive information with the operational data used to generate the measurement plan.

Parameters included are:

Table 2.2: Parameters included in the `Metadata` worksheet.

Parameter	**Description**
Project	Identifies the project or measurement routine associated with the workbook.
Wafer	Identifies the wafer from which the chip originates.
Reticle	Identifies the reticle associated with the chip.
Chip	Identifies the specific chip being characterized.
Operator	Identifies the person responsible for the measurement or configuration.
Date	Records the date associated with the measurement or chip characterization.
Notes	Provides additional information or comments relevant to the measurement routine.

As presented in the following image:

Parameter	Value
Project	Suruga Seiki Hex
Wafer	nan
Reticle	nan
Chip	nan
Operator	
Date	
Notes	Example using normalized device/port model

Figure 2.3: Example of the Metadata worksheet. The table stores general project information using a parameter-value structure, including the project name, wafer, reticle, chip identifier, operator, date, and additional notes.

Devices worksheet

The `Devices` worksheet defines the photonic devices available for characterization. Each row represents a device present on the chip and provides the information required to identify it, classify its type, document its purpose, and determine whether it should be included in the current measurement routine.

The worksheet contains four main columns: `Device`, `Type`, `Description`, and `Measure`. Their purpose is described below.

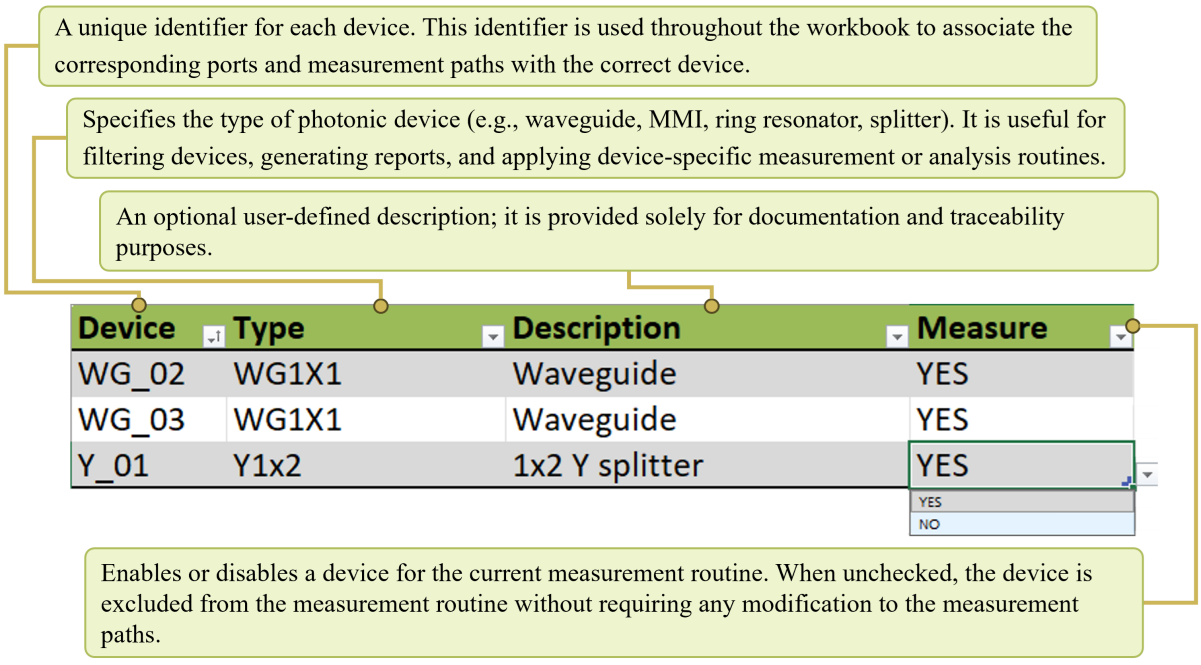


Figure 2.4: Example of the Devices worksheet used by automaper. Each row represents a photonic device and contains its unique identifier, device type, optional description, and measurement-enable status. The Device identifier is used to associate the device with its corresponding ports and measurement paths, while the Measure field determines whether the device is included in the current measurement routine.

- **Device:** This column contains an unique identifier assigned to each device. This identifier is used throughout the workbook to establish the relationship between a device and its corresponding ports and measurement paths. The identifier should therefore remain consistent across all worksheets.

- **Type:** This column specifies the type of the photonic device. This can be used for device-specific filtering, reporting, or future measurement and analysis routines.
- **Description:** This column provides an optional human-readable description of the device. This field is intended primarily for documentation and traceability. It allows the user to provide additional context without affecting the identification or measurement configuration of the device.
- **Measure:** This column determines whether the device is enabled for the current measurement routine. When a device is disabled, all measurement paths associated with that device are excluded from the generated measurement plan. This allows the user to remove an entire device from the current characterization routine without modifying its individual measurement paths.

Ports worksheet

The Ports worksheet defines the optical ports associated with each device and provides the physical information required to locate and characterize them. Each row represents a single port and establishes its relationship with a device through the Device identifier.

In addition to identifying the port, the worksheet specifies its direction, physical coordinates, and coupling configuration. This information is subsequently used by AUTOMAPER to construct the internal chip representation and to resolve the coordinates associated with each measurement path.

Device	Port	Direction	X (um)	Y (um)	Coupling	Comments
WG_02	P1	IN	0	nan	Edge	nan
WG_02	P2	OUT	0	nan	Edge	nan
WG_03	P1	IN	63,5	nan	Edge	nan
WG_03	P2	OUT	63,5	nan	Edge	nan
Y_01	P1	IN	127	nan	Edge	nan
Y_01	P2	OUT	127	nan	Edge	nan
Y_01	P3	OUT	190,5	nan	Edge	nan

Figure 2.5: Example of the Ports worksheet. Each row defines an optical port associated with a specific device, including its identifier, input/output direction, physical coordinates, and coupling configuration. The port coordinates are expressed in micrometers and are referenced to the defined chip coordinate system.

2.2.3 checkPositionsAutomap

Validates the measurement plan generated by AutoMapper before moving the Suruga platform. It calculates and displays the absolute `x1` and `x2` positions for each device defined in the Excel file, but it does not connect to any instrument or move any axes.

Source code: `checkPositionsAutomap.py`.

Workflow

1. Loads the chip layout and measurement plan from an Excel file using `automaper.load_chip()`.
2. Uses `origin_x1` and `origin_x2` as the reference origins.

3. For each measurement, calculates the input and output offsets and the absolute positions of both arms.

4. Displays the device, ports, and calculated positions, and waits for confirmation before displaying the next measurement.

Relevant Configuration

Table 2.3: Relevant configuration parameters for `checkPositionsAutomap`.

Parameter	Initial Value	Description
`EXCEL_FILE`	`chipMaps/SurugaTest.xlsx`	Chip layout and AutoMapper measurement plan.
`origin_x1 / origin_x2`	0.0 μm / 0.0 μm	Reference positions used to calculate the absolute coordinates.

The absolute coordinates are calculated according to

$$x_1 = \text{origin}_{x1} - \text{input}.x \tag{2.1}$$

and

$$x_2 = \text{origin}_{x2} - \text{output}.x. \tag{2.2}$$

Recommended Usage

Run this routine whenever the Excel file, chip coupling, or origin convention changes. Compare the printed positions against the mechanical setup limits before running a routine that moves the machine.

Results

This routine does not create files or perform measurements. Its output consists of the position list displayed in the console.

2.2.4 moveAutomapOPM

Automatically iterates through the measurements defined by AutoMapper, positions the fibers at the input and output waveguides, performs alignment on both arms, and measures the average optical power using the optical power meter (OPM).

Source code: `moveAutomapOPM.py`.

2.2.5 Workflow

1. Loads the measurement plan from the AutoMapper Excel file.

2. Connects to the Suruga platform and creates controllers for `x1`, `x2`, alignment, and the power meter.

3. Loads `origin.txt` or uses the current position as the `WG0` origin.

4. For each device, calculates the absolute positions of both arms and moves `x1` and `x2`.
5. Performs a *flat* alignment on the left arm followed by another alignment on the right arm.
6. Reads ten samples from the OPM, calculates their average, and displays the result in the console.
7. Requests confirmation before continuing and, once all measurements are completed, returns to `WG0`.

2.2.6 Relevant Configuration

Table 2.4: Relevant configuration parameters for `moveAutomapOPM`.

Parameter	Initial Value	Description
`EXCEL_FILE`	`chipMaps/SurugaTest.xlsx`	Chip layout and measurement plan.
`USE_SAVED_ORIGIN / SAVE_ORIGIN`	`True / False`	Control whether `origin.txt` is loaded or saved.
`ORIGIN_FILE`	`origin.txt`	File containing the X1 and X2 coordinates of `WG0`.
`flat_left / flat_right`	`---`	Alignment parameters for each arm.

2.2.7 Results

The average optical power for each device is printed to the console. The routine does not save the measurement values to a file.

2.2.8 Precautions

The routine moves both arms and requires manual confirmation between measurements. Verify the position plan using `checkPositionsAutomap` before running it.

2.2.9 moveAutomapOPM_OSA

Characterizes the devices defined by AutoMapper by combining Suruga platform movement, automatic alignment, an average optical power measurement using the OPM, and spectrum acquisition with the OSA.

Source code: `moveAutomapOPM_OSA.py`.

2.2.10 Workflow

1. Loads the chip layout and measurement plan from Excel.
2. Connects to the Suruga platform, OPM, and OSA, and configures the spectral sweep.
3. Loads or saves the `WG0` origin in `origin2.txt`.
4. For each device and repetition, positions `x1` and `x2` based on the corresponding input and output waveguides.
5. Aligns the left and right arms.

6. Runs an OSA sweep, retrieves the trace, and averages ten power readings.
7. Saves the trace along with the device, ports, optical power, and repetition number as metadata.
8. Returns to `WG0` and closes the OSA connection.

2.2.11 Relevant Configuration

Table 2.5: Relevant configuration parameters for `moveAutomapOPM_OSA`.

Parameter	Initial Value	Description
`DEBUG_MODE`	`True`	Requests manual confirmations; when set to `False`, the routine runs automatically.
`USE_SAVED_ORIGIN / SAVE_ORIGIN`	`False / True`	Control whether the origin stored in `origin2.txt` is loaded or saved.
`EXCEL_FILE`	`chipMaps/SurugaTest01.xlsx`	Chip layout and AutoMapper measurement plan.
`num_repeticiones`	`1`	Number of complete passes through the measurement plan.
`CENTER / SPAN`	`1550 nm / 90 nm`	OSA spectral window.
`RESOLUTION / SENSITIVITY`	`0.2 nm / -65 dBm`	OSA resolution and sensitivity.
`channel`	`1`	Power meter channel.

2.2.12 Results

For each device, the routine saves a trace to:

`data/<date>/medicion_<repetition>_<device>_<input>_<output>.txt`

The file contains the spectral data and metadata including the device, ports, average optical power, and repetition number.

2.2.13 Precautions

With `DEBUG_MODE=True`, the routine pauses before the OSA sweep, between devices, and before returning to the origin.

Before using automatic mode, validate the movements with `checkPositionsAutomap`, verify the origin, and confirm that the OSA VISA resource is available.

Appendix A

TwinCAT XAE (eXtended Automation Engineering) Installation

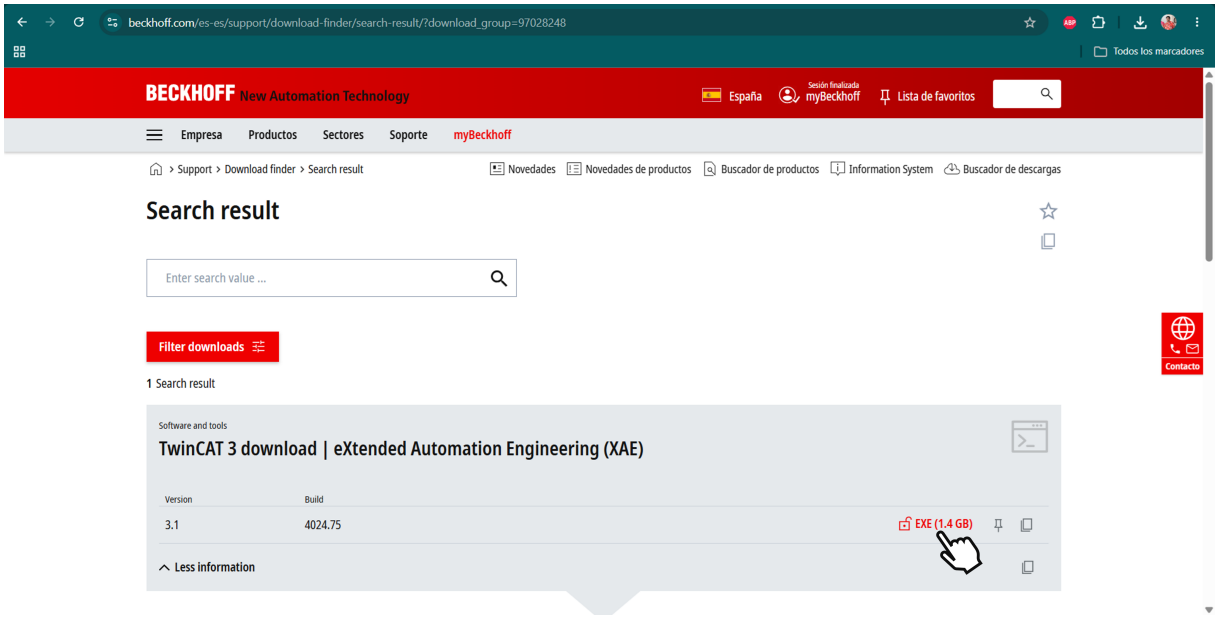


Figure A.1: Beckhoff Download Finder showing the TwinCAT 3 eXtended Automation Engineering (XAE) package available for download.

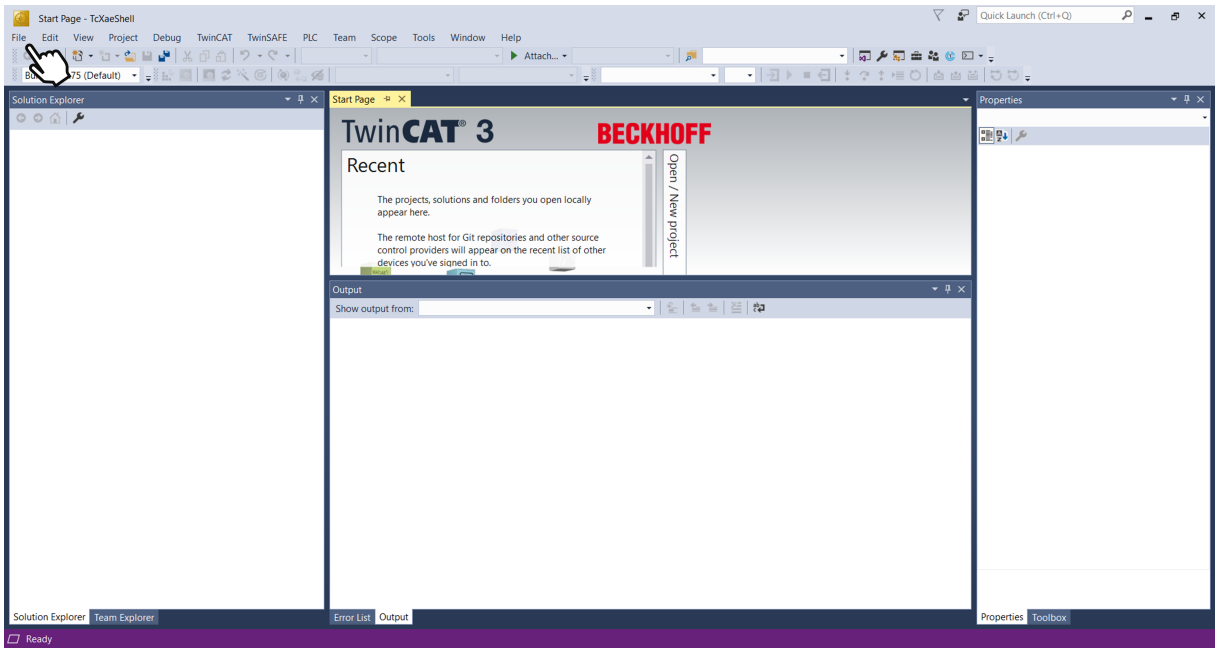


Figure A.2: TwinCAT 3 XAE start page after launching the development environment.

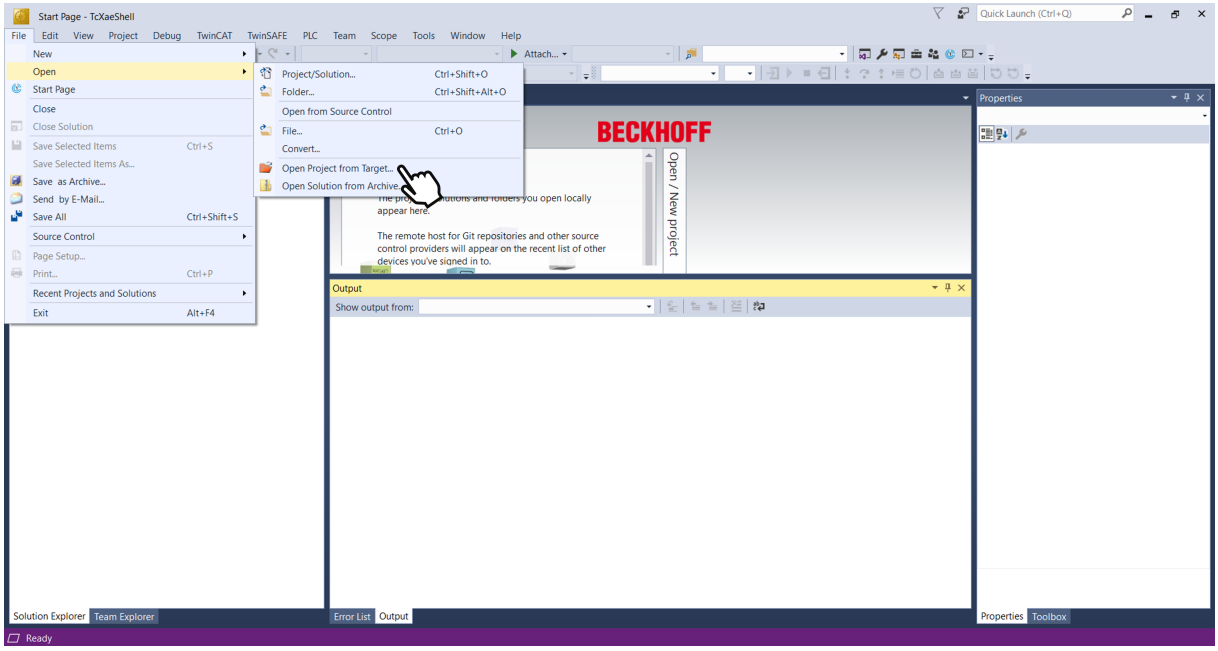


Figure A.3: Selection of the `Open Project from Target` option from the TwinCAT 3 `File` menu.

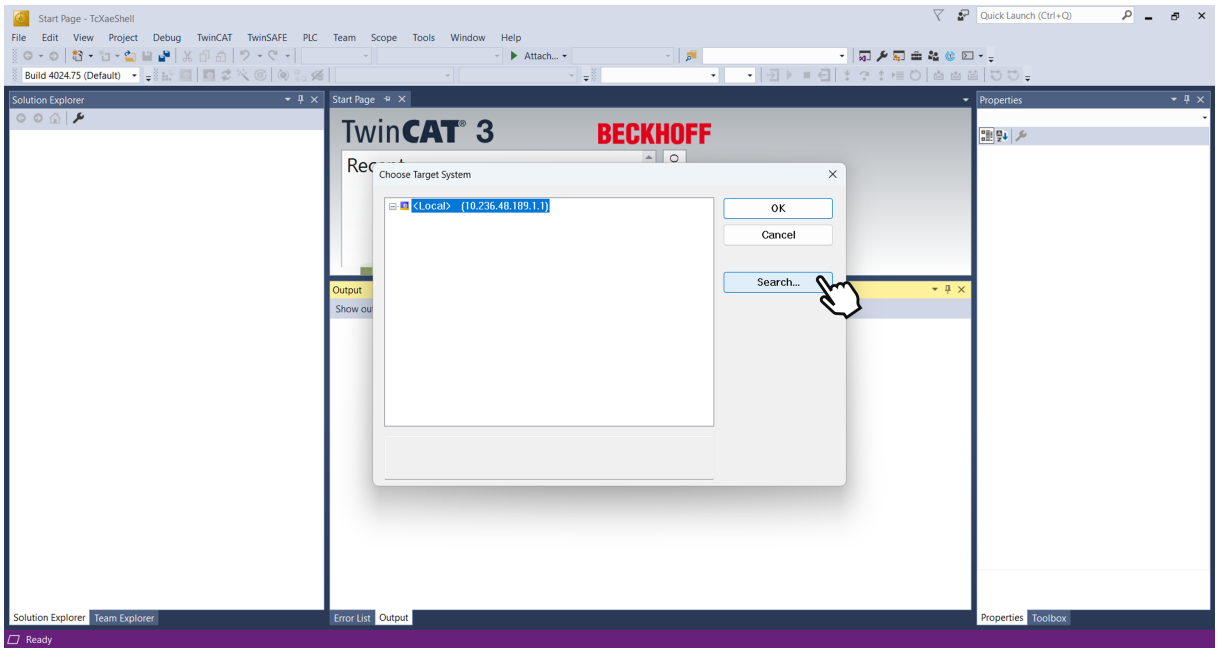


Figure A.4: Target System dialog for selecting the TwinCAT system to be accessed.

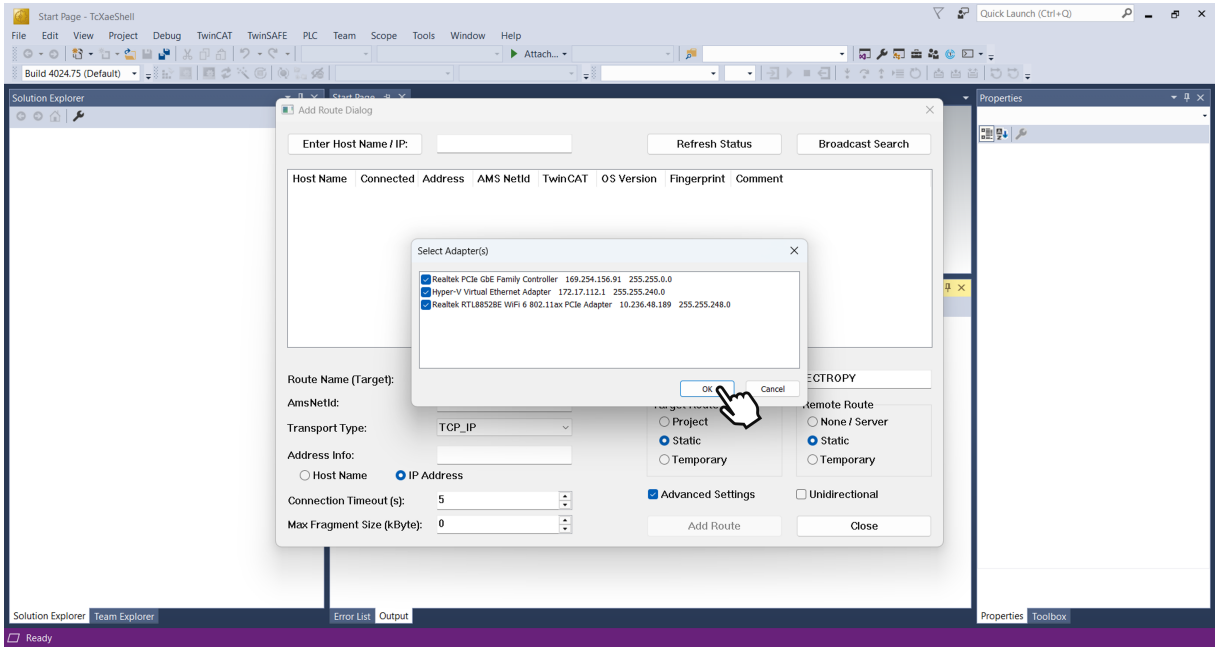


Figure A.5: Selection of the network adapter used to establish the connection with the TwinCAT target system.

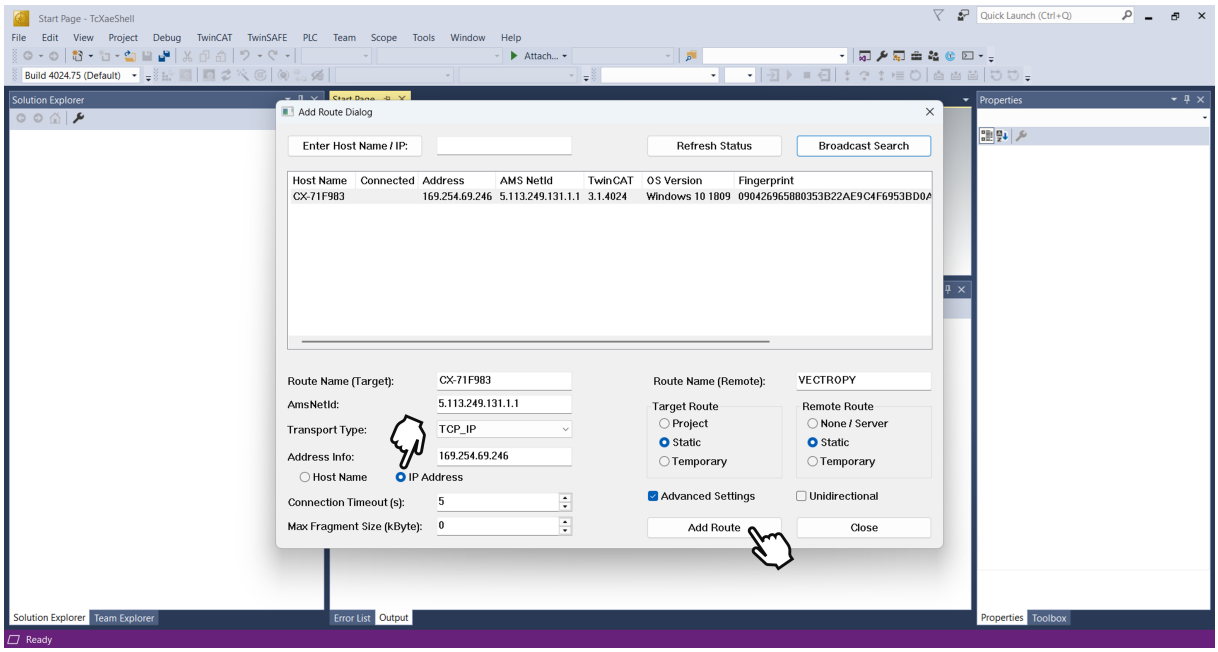


Figure A.6: Configuration of the TwinCAT remote route using the detected target system and its AMS Net ID and IP address.

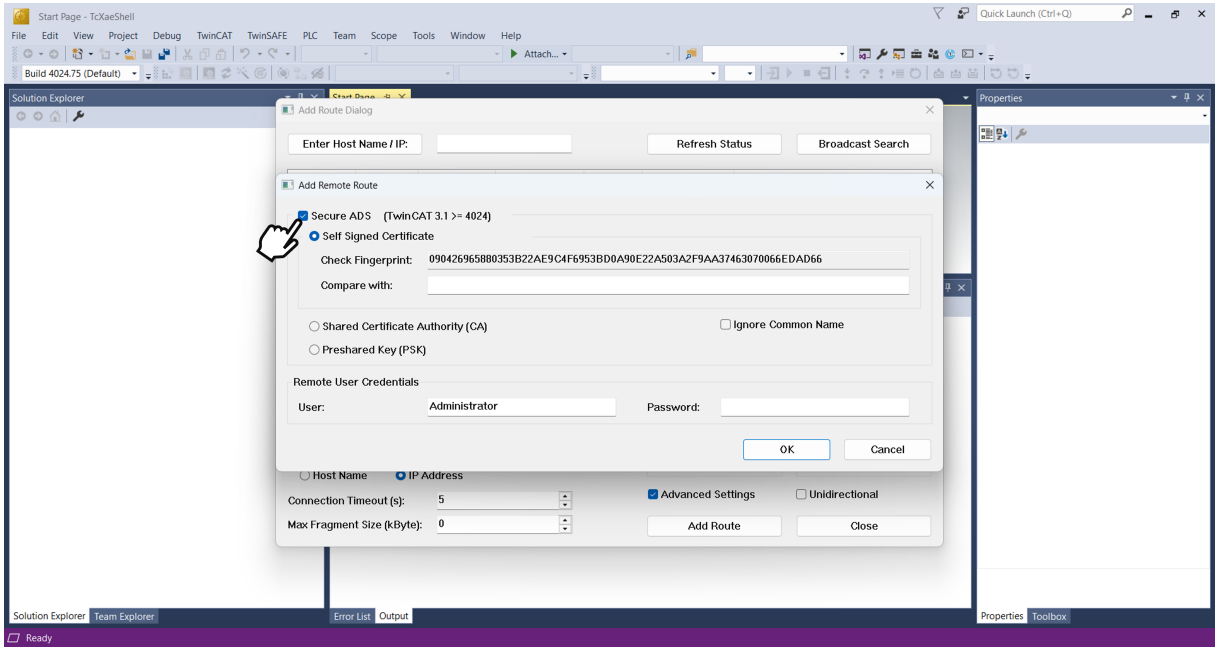


Figure A.7: Secure ADS configuration using the target system's self-signed certificate and fingerprint.

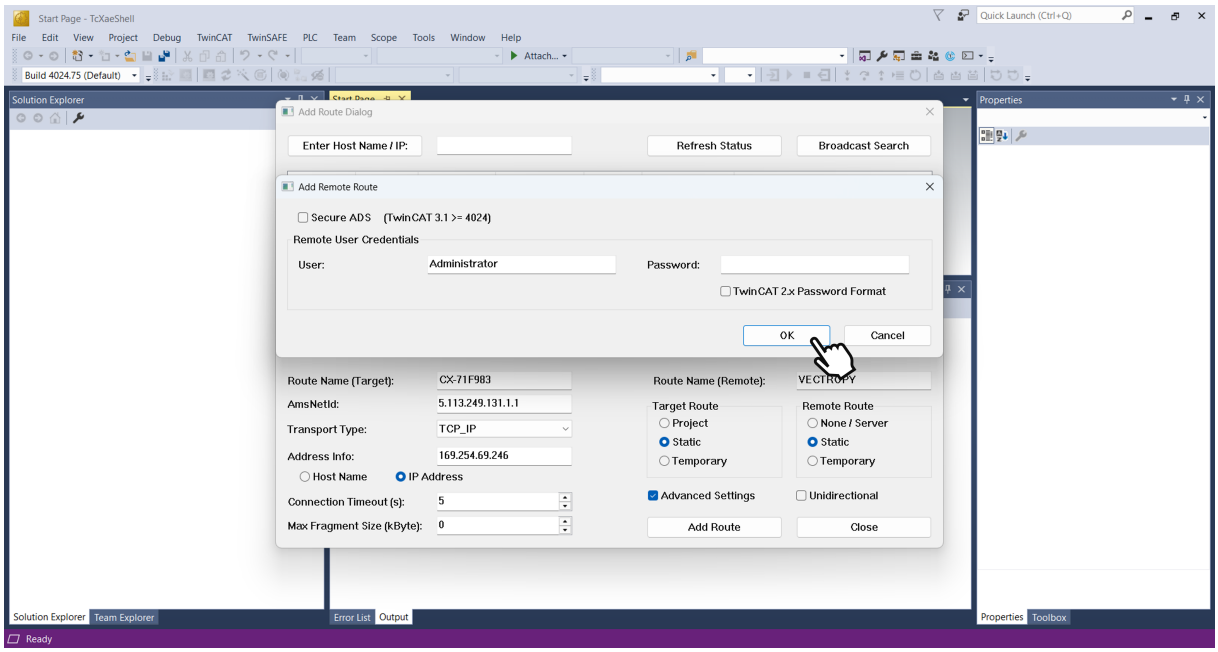


Figure A.8: Configuration of the remote user credentials for the TwinCAT target system.

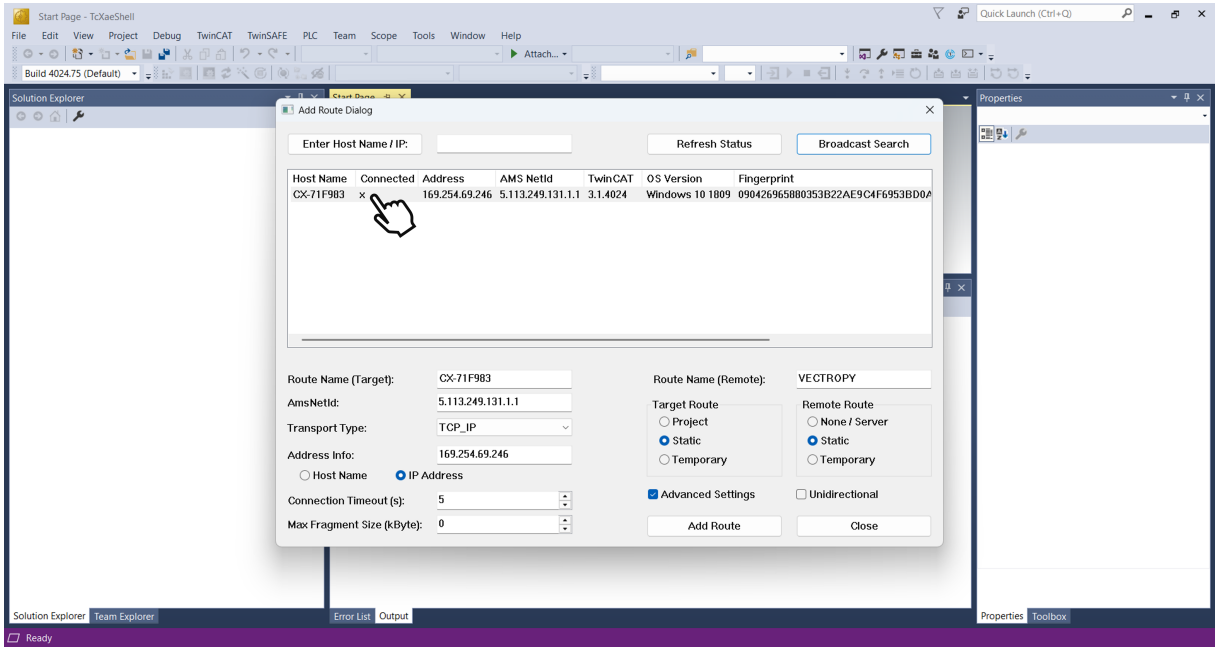


Figure A.9: Configured remote route showing the target system, AMS Net ID, IP address, and route settings.

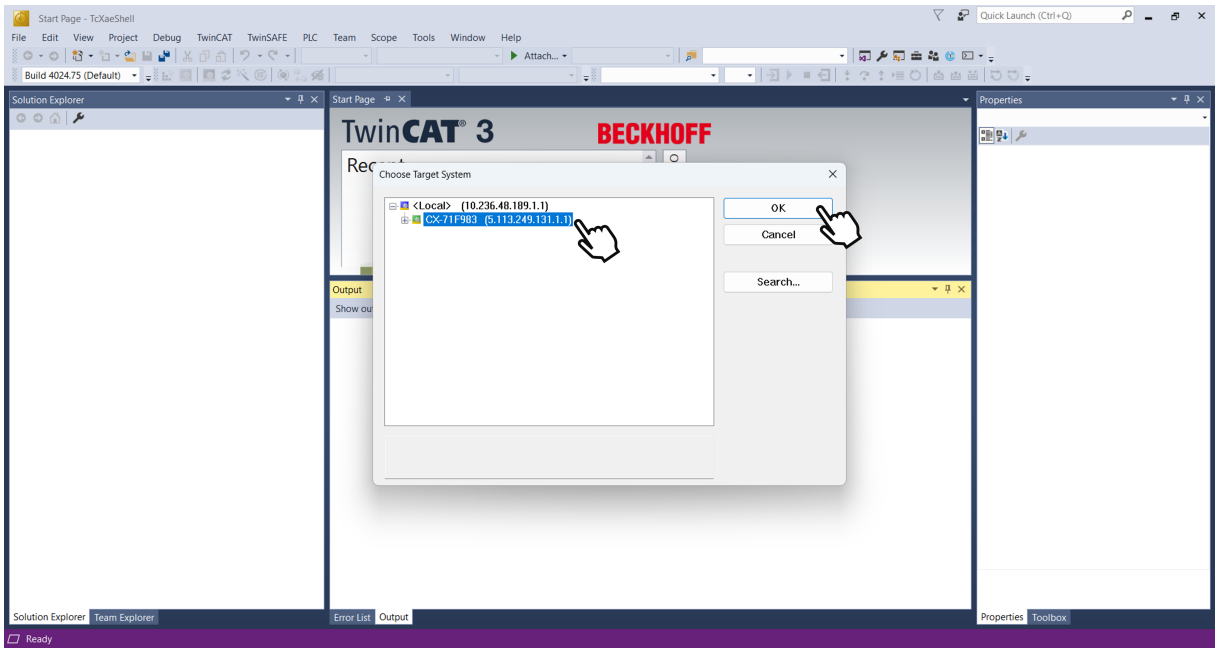


Figure A.10: Selection of the configured TwinCAT target system for the project.

Appendix B

Suruga Seiki API Documentation

To interact with the Suruga Seiki platform from an external application, the corresponding .NET assembly must first be loaded. An assembly is a compiled unit of code that contains the classes, types, and functionality required by the application to interact with the Suruga Seiki system. The assembly exposes a set of classes that represent different components and functionalities of the platform.

From an object-oriented programming perspective, a class defines the structure and behavior of a particular type of object. Its methods represent the operations or actions that can be performed, while its properties describe characteristics or values associated with its objects. An object is a concrete instance of a class. As an analogy, a class can be understood as a construction blueprint, while an object is a concrete instance created from that blueprint.

The classes exposed by the Suruga Seiki .NET API generally follow a common structure. Each class provides a constructor that allows an instance of the corresponding class to be created and initialized, providing access to its associated functionality. The classes also expose a set of methods that allow the application to perform specific operations, interact with the platform, and retrieve information.

In addition to constructors and methods, the API uses nested types to organize and facilitate the interaction with related data. These nested types can be used to define a finite set of predefined values through enumerations or to group multiple related values into a structured data type. This organization allows related pieces of information to be handled as a single entity rather than as individual parameters.

B.1 Profile class

The `SurugaSeiki.Motion.Profile` class illustrates this structure. Its methods provide the operations required to configure, control, and monitor the profiling process. Some methods interact with the nested types by receiving them as input parameters or returning them as output values. For example, `Start()` and `Stop()` return an `ErrorCode`, `GetProfileStatus()` returns a `Status`, while `SetProfile()` receives a `ProfileParameter` and `RequestProfileData()` returns a `ProfileData`.

The nested types associated with the `Profile` class support different aspects of the profiling functionality. The `ProfileParameter` and `ProfileData` types are structured data types used to encapsulate, respectively, the profiling configuration parameters and the acquired measurement data. In contrast, `Status` and `ErrorCode` are enumerations that define a finite set of predefined values representing the current state of the profiling operation and the error conditions returned

by the API.

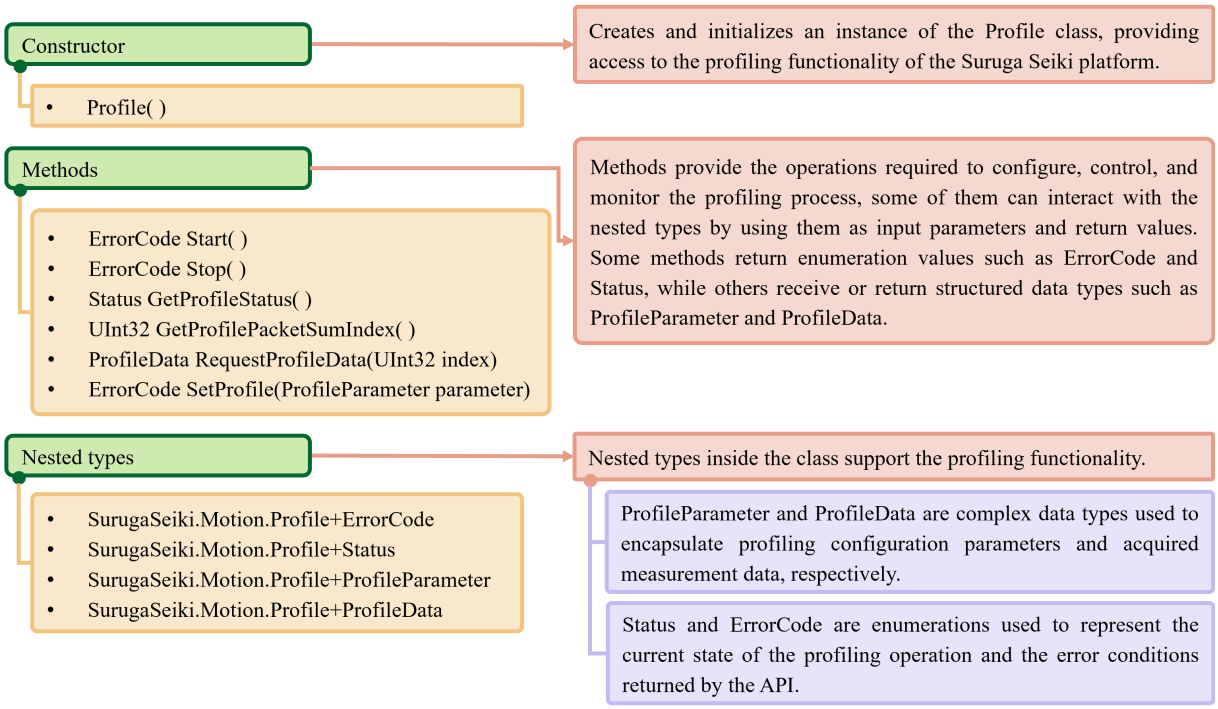


Figure B.1: Structure of the `Profile` class, showing its constructor, profiling methods, and nested types for configuration parameters, acquired data, status information, and error codes.

```
================================================================================================
SurugaSeiki.Motion.Profile
================================================================================================

Constructor:
  Profile()

Methods:
  ErrorCode Start()
  ErrorCode Stop()
  Status GetProfileStatus()
  UInt32 GetProfilePacketSumIndex()
  ProfileData RequestProfileData(UInt32 index)
  ProfileData RequestProfileData(UInt32 index, Boolean isRotationCenterShift)
  ErrorCode SetProfile(ProfileParameter parameter)
  ErrorCode SetProfile(ProfileParameter parameter, RotationCenter rotationCenter)

Nested Types:
  SurugaSeiki.Motion.Profile+ErrorCode
  SurugaSeiki.Motion.Profile+Status
  SurugaSeiki.Motion.Profile+ProfileParameter
  SurugaSeiki.Motion.Profile+ProfileData

================================================================================================
SurugaSeiki.Motion.Profile+ErrorCode
================================================================================================

Fields:
  Int32 value__
  static ErrorCode None
  static ErrorCode Axis
  static ErrorCode Profiling
  static ErrorCode Parameter

================================================================================================
SurugaSeiki.Motion.Profile+ProfileData
================================================================================================

Fields:
  UInt16 mainAxisNumber
  UInt16 sub1AxisNumber
  UInt16 sub2AxisNumber
  UInt16 signalCh1Number
  UInt16 signalCh2Number
  UInt32 packetIndex
  UInt32 dataCount
  Double[] mainPositionList
  Double[] sub1PositionList
  Double[] sub2PositionList
  Double[] signalCh1List
```

```
  Double[] signalCh2List

================================================================================================
SurugaSeiki.Motion.Profile+ProfileParameter
================================================================================================

Fields:
  UInt16 mainAxisNumber
  UInt16 sub1AxisNumber
  UInt16 sub2AxisNumber
  UInt16 signalCh1Number
  UInt16 signalCh2Number
  Double mainRange
  Double sub1Range
  Double sub2Range
  Double speed
  Double accelRate
  Double decelRate
  UInt32 smoothing

================================================================================================
SurugaSeiki.Motion.Profile+Status
================================================================================================

Fields:
  Int32 value__
  static Status Stopping
  static Status Success
  static Status Profiling
  static Status ProfileDataOver
  static Status InvalidParameter
  static Status ServoIsNotReady
  static Status ServoIsAlarm
  static Status StageOnLimit
  static Status TorqueLimit
```

B.2 System class

Having introduced the general structure of the Suruga Seiki .NET API through the `Profile` class, we will now review the `System` class, which provides a broader interface to the Suruga Seiki platform. While `Profile` is focused on the configuration, execution, and monitoring of profiling operations, `System` provides access to general system-level information and functionality.

Unlike the other API classes described in this section, the `System` class does not expose a public constructor in the API interface. Instead, the API provides access to a system instance through the `Instance` property. This instance is then used by the Python wrapper developed in this work to configure and communicate with the Suruga Seiki controller.

Overall, the `System` class provides the basic functions required to establish and monitor communication with the Suruga Seiki controller. Its functionality can be grouped into three main aspects: connection management through `GetInstance()`, `SetAddress()`, and `Connected`; system error monitoring through `IsError()` and `GetErrorCode()`; and safety-state monitoring through `IsEmergencyAsserted()`. These functions provide the foundation on which the higher-level classes and routines used by the application can interact with the controller.

```
================================================================================================
SurugaSeiki.Motion.System
================================================================================================

Properties:
  UInt16 MaxAxisStageNumber (get/set)
  UInt16 MaxAnalogInputChNumber (get/set)
  UInt16 MaxAlignmentNumber (get/set)
  UInt16 MaxAngleAdjustmentNumber (get/set)
  UInt16 MaxDInChNumber (get/set)
  UInt16 MaxDOutChNumber (get/set)
  UInt16 MaxAnalogOutputChNumber (get/set)
  UInt16 MaxPowerMeterChNumber (get/set)
  System Instance (get)
  Boolean Connected (get)
  String DllVersion (get)
  String SystemVersion (get)

Methods:
  Boolean IsError()
  ErrorCode GetCurrentErrorCode()
  Boolean IsEmergencyAsserted()
  Void SetAddress(String addr)
  Void DisconnectAds()

Fields:
```

```
  static UInt16 MaxProfileNumber
  static UInt32 ProfileDataPacketMaxDataNumber

Nested Types:
  SurugaSeiki.Motion.System+ErrorCode

================================================================================================
SurugaSeiki.Motion.System+AlignmentChParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentCommand
================================================================================================

Fields:
  Int32 value__
  static AlignmentCommand Null
  static AlignmentCommand StartSingle
  static AlignmentCommand StartFlat
  static AlignmentCommand StartFocus
  static AlignmentCommand StartTwoChRotation
  static AlignmentCommand StartMMFRotation
  static AlignmentCommand Stop
  static AlignmentCommand SetSingle
  static AlignmentCommand SetFlat
  static AlignmentCommand SetFocus
  static AlignmentCommand SetTwoChRotation
  static AlignmentCommand SetMMFRotation
  static AlignmentCommand StartLoopBack
  static AlignmentCommand SetLoopBack

================================================================================================
SurugaSeiki.Motion.System+AlignmentMainAxisParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentPowerMeterInfo
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentStatusInfo
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AnalogOutputControlCommand
================================================================================================

Fields:
  Int32 value__
  static AnalogOutputControlCommand Null
  static AnalogOutputControlCommand SetAnalogOutput

================================================================================================
SurugaSeiki.Motion.System+AngleAdjustmentCommand
================================================================================================

Fields:
  Int32 value__
  static AngleAdjustmentCommand Null
  static AngleAdjustmentCommand SetParameter
  static AngleAdjustmentCommand Start
  static AngleAdjustmentCommand Stop

================================================================================================
SurugaSeiki.Motion.System+AngleAdjustmentParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AngleAdjustmentStatusInfo
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+Axis2DCommand
================================================================================================

Fields:
  Int32 value__
  static Axis2DCommand Null
  static Axis2DCommand MoveAbsolute
  static Axis2DCommand MoveRelative
  static Axis2DCommand MoveAbsoluteAngle
  static Axis2DCommand MoveRelativeAngle
  static Axis2DCommand Stop
  static Axis2DCommand SetCompositionSpeed
  static Axis2DCommand SetAccelRate
  static Axis2DCommand SetDecelRate
  static Axis2DCommand SetStageNumber

================================================================================================
SurugaSeiki.Motion.System+Axis2DParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AxisCommandStruct
================================================================================================
```

```
================================================================================================
SurugaSeiki.Motion.System+AxisComponentCommand
================================================================================================

Fields:
  Int32 value__
  static AxisComponentCommand Null
  static AxisComponentCommand MoveAbsolute
  static AxisComponentCommand MoveRelative
  static AxisComponentCommand ReturnOrigin
  static AxisComponentCommand Stop
  static AxisComponentCommand TurnOnServo
  static AxisComponentCommand TurnOffServo
  static AxisComponentCommand ResetError
  static AxisComponentCommand SetMaxSpeed
  static AxisComponentCommand SetAccelRate
  static AxisComponentCommand SetDecelRate
  static AxisComponentCommand SetSineMotion
  static AxisComponentCommand SetFulcrumSpan
  static AxisComponentCommand SetEncoderStepLength
  static AxisComponentCommand SetCWTorquelimit
  static AxisComponentCommand SetCCWTorquelimit

================================================================================================
SurugaSeiki.Motion.System+AxisControlType
================================================================================================

Fields:
  Int32 value__
  static AxisControlType AxisComponents
  static AxisControlType Axis2D
  static AxisControlType Axis3D

================================================================================================
SurugaSeiki.Motion.System+AxisMethod
================================================================================================

Fields:
  Int32 value__
  static AxisMethod Null
  static AxisMethod MoveAbsolute
  static AxisMethod MoveRelative
  static AxisMethod MoveAbsoluteAngle
  static AxisMethod MoveRelativeAngle
  static AxisMethod Stop
  static AxisMethod SetCompositionSpeed
  static AxisMethod SetAccelRate
  static AxisMethod SetDecelRate
  static AxisMethod SetStageNumber
  static AxisMethod SetSubStageNumber
  static AxisMethod SetRotationCenter
  static AxisMethod SetFixedAxisOffset

================================================================================================
SurugaSeiki.Motion.System+AxisStatus
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+DOutControlCommand
================================================================================================

Fields:
  Int32 value__
  static DOutControlCommand Null
  static DOutControlCommand SetDout
  static DOutControlCommand SetPulseOutput

================================================================================================
SurugaSeiki.Motion.System+DeviceControlCommand
================================================================================================

Fields:
  Int32 value__
  static DeviceControlCommand Null
  static DeviceControlCommand SetWaveLength

================================================================================================
SurugaSeiki.Motion.System+DeviceStatusMap
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+ErrorCode
================================================================================================

Fields:
  Int32 value__
  static ErrorCode NoError
  static ErrorCode ConnectionLost
  static ErrorCode VersionError
  static ErrorCode MaxNumberOfErrorCode

================================================================================================
SurugaSeiki.Motion.System+Point2d
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+PowerMeterControlCommand
================================================================================================
```

```
Fields:
  Int32 value__
  static PowerMeterControlCommand None
  static PowerMeterControlCommand SetRange
  static PowerMeterControlCommand SetWaveLength
  static PowerMeterControlCommand SetCommunicationPaused

================================================================================================
SurugaSeiki.Motion.System+ProfileCommand
================================================================================================

Fields:
  Int32 value__
  static ProfileCommand Null
  static ProfileCommand SetParameter
  static ProfileCommand Start
  static ProfileCommand Stop

================================================================================================
SurugaSeiki.Motion.System+ProfileParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+RotationCenterParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+SystemInfo
================================================================================================
```

Bibliography